University of Saarland
Algorithmic Foundations & Cryptography

Bachelor's Thesis

FROM EID TO WALLET: USER-FRIENDLY, AGE-COMPLIANT AND PRIVACY-PRESERVING SMART ACCOUNTS

Author: Simonas Kanapeckas
Student ID: 7045887
Supervisor: Dr. Lucjan Hanzlik
Date: May 20, 2026

Abstract

ERC-4337 account abstraction introduces a new execution model for Ethereum in which transactions are replaced by *UserOperations* – arbitrary objects validated by user-deployed smart contracts rather than the protocol itself. This decoupling of authentication logic from the underlying chain makes it possible to replace the traditional single-key signing model with any cryptographic mechanism that can be expressed in Solidity, opening the door to biometric authenticators, hardware identity tokens, and zero-knowledge proofs as wallet authentication methods. This thesis exploits this flexibility to build and evaluate a five-level progressive wallet framework that addresses three limitations of existing blockchain wallets: poor usability due to private key management, loss of self-custody in corporate custodial solutions, and the absence of privacy-preserving compliance mechanisms. The five authentication levels are: (1) a baseline secp256k1 ECDSA wallet, (2) WebAuthn biometric authentication using device-native passkeys, (3) Lithuanian government eID card authentication requiring on-chain secp384r1 ECDSA verification, (4) a zero-knowledge circuit that proves age compliance from a government-signed eID certificate without disclosing the underlying birth date, and (5) a combined system in which WebAuthn handles routine transactions, the eID card serves as the recovery mechanism for a lost authenticator, and the ZK proof ensures age-compliance without disclosing the personal data of the user. The results show that ERC-4337 is expressive enough to encode government-grade identity verification and compliance logic directly at the wallet level without any off-chain intermediary, while the gas analysis reveals the practical limits of deploying such mechanisms on current EVM-compatible networks. Discovered security limitations of the implementation, are discussed in the final chapter.

Contents

1 Introduction **4**

2 Related Work **5**

2.1 Smart Contract Wallets and Account Abstraction 5

2.2 On-Chain Elliptic Curve Signature Verification 6

2.3 Zero-Knowledge Proofs for Verification of Personal Information 6

3 Background **6**

3.1 Ethereum and Account Abstraction 6

3.1.1 ERC-4337 Account Abstraction 8

3.2 Elliptic Curve Cryptography 10

3.2.1 Prerequisites: Finite Fields and Abelian Groups 10

3.2.2 Elliptic Curve over a Finite Field 10

3.2.3 Elliptic Curve Cryptography 11

3.2.4 Elliptic Curve Digital Signature Algorithm 12

3.2.5 secp256k1 – Ethereum Native Signatures 14

3.2.6 secp256r1 – WebAuthn Signatures 16

3.2.7 WebAuthn procedure, parties and terminology. 17

3.2.8 secp384r1 – Lithuanian eID Signatures 19

3.3 Zero-Knowledge Proofs 20

3.3.1 Groth16 22

3.3.2 Circom and SnarkJS 23

4 Implementation of Smart Accounts **23**

4.1 Basic ECDSA Account 24

4.2 WebAuthn Biometric Account 25

4.3 Lithuanian eID Account 26

4.4 Adding Zero-Knowledge Age Verification to the eID Account 27

4.4.1 Problem Statement and Design Goals 27

4.4.2 Certificate Parsing in the Circuit 28

4.4.3 secp384r1 Signature Verification in the Circuit 29

4.4.4 Age Constraint 29

4.4.5 Soundness and Zero-Knowledge Arguments 30

4.4.6 Trusted Setup and Verifier Contract 30

4.5 Combined Final Multi-Factor Account 31

5 Evaluation **34**

5.1 Testing Environment 34

5.2 Evaluation Metric: Gas Cost 34
5.3 Results 34
5.4 Limitations 36
5.4.1 Identity Binding at Account Creation 36
5.4.2 Recovery Dependency on eID Card Availability 36

6 Conclusion **36**

1 Introduction

Public blockchain networks such as Ethereum enable individuals to hold and transfer digital assets without relying on any central institution. The property that makes this possible is *self-custody*: whoever controls a cryptographic private key controls the associated assets unconditionally and without appeal. In practice, however, most users avoid this model entirely – delegating custody to corporate exchanges such as Binance or Coinbase, who hold the keys on their behalf – a trade-off the crypto community captured in a single phrase: *not your keys, not your coins* – or struggle with the fragility of managing raw private keys themselves. The collapse of the FTX exchange in 2022 [1] illustrates the custodial risk at scale with more than 8 billion dollars worth of user assets frozen, while estimates of permanently lost cryptocurrency attributable to forgotten, stolen or destroyed keys run into the hundreds of billions of dollars as well [2]. In both cases the root cause is the same: the dominant account model on Ethereum, the Externally Owned Account (EOA), is defined entirely by a single key with no recovery mechanism [3], no graduated authentication, and no way to enforce compliance constraints at the wallet level.

ERC-4337 account abstraction [4] offers a principled path beyond this limitation. Rather than issuing transactions signed by a fixed protocol key, users submit *UserOperations* – structured objects validated by smart contracts they deploy and control. This moves authentication logic off the protocol layer and into user space, making it possible to replace the private key with any cryptographic mechanism expressible in Solidity – the programming language used to write Ethereum smart contracts [5]. The standard has attracted significant implementation work, yet most deployments use it only to add convenience features such as gas sponsorship or batching. The ERC-4337 standard is expressive enough to support alternative signature schemes, and prior work has demonstrated WebAuthn passkey authentication within the framework [6]. Whether it can be extended to government-grade authentication – specifically, on-chain verification of national eID card signatures and zero-knowledge compliance proofs derived from government-signed certificates – within a single coherent wallet architecture remains an open question.

The regulatory context sharpens the problem further. The European Union's revised eIDAS 2.0 regulation [7] requires member states to issue digital identity wallets that let citizens prove specific attributes about themselves – such as age or nationality – without revealing anything else. At the same time, the Markets in Crypto-Assets regulation (MiCA) [8] requires crypto-asset service providers to verify who their users are. Both regulations assume that a digital identity can be linked to a real person. Blockchain wallets, however, are just a public key and an address – there is no name, no age, no legal identity attached. Existing services bridge this gap by sending users through an off-chain identity check with a third-party provider, who then vouches for them. This works, but it simply moves the trust problem: instead of trusting a centralised exchange with your

assets, you now trust a centralised identity provider with your personal data.

What is missing is a wallet that carries its own proof. Governments already issue digital identity cards that certify who you are with legally binding guarantees, zero-knowledge cryptography already makes it possible to prove a fact about those credentials – such as being over 18 – without revealing anything else. The gap is not a lack of ideas but a lack of compound technologies which unify these separate tools. The smart account layer that would connect them, have been built separately, and joining them is hard. Government identity cards use cryptographic curves and certificate formats that Ethereum was never built to handle efficiently, the circuits needed to verify those certificates inside a zero-knowledge proof push the limits of what a browser can reasonably compute. This thesis is an attempt to connect all mentioned components together.

In the following chapters, we address this question by designing, implementing, and evaluating a five-level progressive wallet framework built on ERC-4337. Each level introduces a more demanding authentication mechanism, finalising with a system that combines biometric authentication, Lithuanian government eID card signatures, and zero-knowledge age verification derived from government-signed certificates. The framework is evaluated for functional correctness, on-chain gas cost, security, and privacy. We also identify and discuss the limitations of the current design, including the inherent tension between hardware key security and account recoverability.

2 Related Work

2.1 Smart Contract Wallets and Account Abstraction

The limitations of Externally Owned Accounts have been recognised since Ethereum's early days. EIP-86 [9] first proposed moving authentication logic into user-deployed forwarding contracts in 2017 that could validate transactions using arbitrary signature schemes instead of the hard-coded ECDSA mechanism. EIP-86 required a hard fork and was never included in any network upgrade and was considered stagnant. EIP-2938 [10] revisited the idea in 2020 by defining a new transaction type – `AA_TX_TYPE` – via the EIP-2718 typed transaction envelope [11], in which smart contracts could act as top-level accounts, validating and paying for their own transactions using a new `PAYGAS` opcode. ERC-4337 [4] later inherited the same core idea – user intents with arbitrary validation logic – but abandoned the typed transaction approach entirely in favour of a higher-layer mempool, and a global EntryPoint contract, allowing UserOperations to be validated by arbitrary smart contract logic without any consensus modifications. This thesis extends ERC-4337 into government-grade authentication territory, integrating secp384r1 eID signatures and zero-knowledge age verification into a single wallet architecture – a direction none of the existing implementations have pursued.

2.2 On-Chain Elliptic Curve Signature Verification

The EVM natively supports only secp256k1 ECDSA through the `ecrecover` precompile. Verifying signatures from any other curve requires a pure-Solidity implementation, which is expensive because 256-bit EVM arithmetic does not map cleanly onto larger field sizes. For P-256 (secp256r1), OpenZeppelin's Contracts library provides a verifier that calls the RIP-7212 precompile [12] when available and falls back to a pure-Solidity implementation otherwise [6]. This thesis uses the OpenZeppelin library directly for WebAuthn signature verification. For P-384 (secp384r1), no precompile exists or is proposed. The most directly relevant prior work is the Ethstonia project [13], an implementation that first demonstrated secp384r1 ECDSA verification in Solidity by implementing 384-bit arithmetic from scratch. This thesis builds directly on Ethstonia's Solidity contracts, extending them into the ERC-4337 `validateUserOp` flow and integrating the full off-chain pipeline from eID card reader interaction to on-chain verification.

2.3 Zero-Knowledge Proofs for Verification of Personal Information

Zero-knowledge proofs have attracted significant attention as a mechanism for on-chain identity verification. Polygon ID [14] allows users to prove attributes derived from verifiable credentials without revealing the underlying data, using Circom-based zkSNARKs. ZKPassport [15] enables privacy-preserving identity verification from government-issued passports and national ID cards: a user scans the NFC chip of their document on a smartphone, and the application generates a zero-knowledge proof that a specific claim — such as age or nationality — holds according to the data signed by the issuing government, without revealing any other personal information. This thesis follows the same trust model, anchoring the age proof directly to the Lithuanian government CA's P-384 signature verified inside the circuit. Two open-source libraries make this feasible: the `asn1-parser-circom` library [16] provides Circom templates for parsing DER-encoded ASN.1 structures, enabling the personal code to be extracted from the certificate inside the circuit, and the Curve384 Circom library [17] provides P-384 elliptic curve arithmetic in Circom required to verify that signature.

3 Background

3.1 Ethereum and Account Abstraction

Ethereum is a decentralised, programmable blockchain network in which the global state – a mapping from account addresses to account data – is updated by executing transactions [3].

Its formal specification, the Ethereum Yellow Paper, defines the protocol as a transaction-based state machine: beginning from a genesis state, each valid transaction drives the network from one state to the next [18]. Unlike Bitcoin, which treats the blockchain as a ledger of value transfers, Ethereum equips each account with the ability to run arbitrary code through the *Ethereum Virtual Machine* (EVM), a stack-based execution environment whose computation is bounded by *gas* – a unit of computational cost that prevents unbounded execution and compensates validators.

Ethereum distinguishes two kinds of accounts. An *Externally Owned Account* (EOA) is controlled by a secp256k1 private key. It has a balance and a nonce, but no code. A *contract account* is deployed by a transaction, has an immutable code field, and executes that code whenever it gets called. Every transaction on the network must originate from an EOA – only EOAs can initiate state changes [18] – and every transaction is authenticated by an ECDSA signature produced with the EOA's private key.

A transaction contains a type, recipient address, an optional ETH value, a data payload, a gas limit, a nonce to prevent replay attacks, and three signature components (r, s) and a recovery identifier v [18, 19]. Before including a transaction in a block, each network node verifies the signature by recovering the sender's public key from (v, r, s) and the transaction hash using the `ecrecover` operation – a public-key recovery algorithm defined in the Yellow Paper [18] and formally described in section 3.2.5 – then deriving the sender's address as the rightmost 20 bytes of the Keccak-256 hash of that public key. If the recovered address matches the declared sender and the account has sufficient balance to cover the gas, the transaction is valid. The EVM then executes the transaction, updating the world state atomically: either all state changes commit, or the transaction reverts entirely and only the gas fee is deducted.

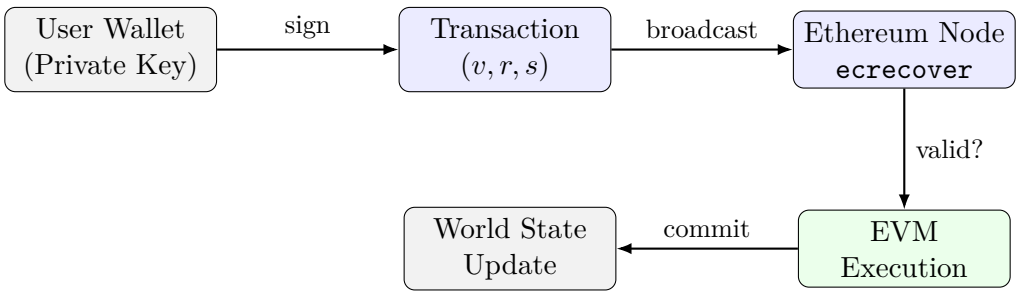


Figure 1: Ethereum transaction lifecycle: a user signs a transaction with their private key, broadcasts it to the network, and the receiving node verifies the ECDSA signature via `ecrecover` before executing the transaction in the EVM and committing the resulting state change.

The `ecrecover` operation is exposed as a precompile in the EVM, making it available to smart contracts with a fixed gas cost. It is the only native signature verification mechanism in the protocol: the EVM natively understands only secp256k1 ECDSA. Any other cryptographic scheme – secp256r1 for WebAuthn, secp384r1 for government

eID cards, or operations for zero-knowledge proof verification – must be implemented in Solidity or via additional precompiles. This constraint is central to the engineering challenges addressed in this thesis.

It is worth noting that not all transactions trigger EVM execution. A simple ETH transfer to an EOA is handled entirely by the state transition function, which updates the sender and recipient balances directly without invoking the EVM. The execution path shown in fig. 1 applies specifically to transactions whose recipient is a contract account, which is the case for all wallet interactions in this thesis.

3.1.1 ERC-4337 Account Abstraction

As established in the preceding section, every Ethereum transaction must originate from an EOA authenticated by a secp256k1 signature. This makes the authentication scheme a protocol-level constant rather than an application-level choice. ERC-4337 breaks this constraint without any changes to the consensus layer by introducing a parallel transaction processing pipeline in which user intents are expressed as *UserOperations* – structured objects that are validated not by the protocol but by user-deployed smart contracts [4].

UserOperation. A `UserOperation` is a data structure containing the address of account who created the user operation, a nonce, optional deployment factory address and bytecode for the factory (`initCode`), a call payload, gas parameters, and a `signature` field whose contents and interpretation are entirely defined by the wallet contract [4]. There is no protocol-enforced requirement that `signature` contain a secp256k1 signature – it can carry a WebAuthn signature, a eID signature, be a threshold signature or any other cryptographic signature.

EntryPoint. The global singleton `EntryPoint` contract is the coordinator of the ERC-4337 system. It receives batches of `UserOperation`s from bundlers, calls `validateUserOp` on each sender wallet to verify authenticity and reserve gas payment, and then dispatches the operations for execution. The `EntryPoint` provides the security boundary: wallets may only be called through it, and the validation and execution phases are strictly separated to prevent reentrancy attacks [4].

validateUserOp. Each wallet contract must implement the `validateUserOp` function, which receives the full `UserOperation` and its signed hash, and returns a validation result and a time validity window. All custom authentication logic lives here – the wallet inspects the `signature` field and accepts or rejects the operation according to whatever cryptographic scheme it implements. This is the sole integration point for all five authentication levels in this thesis [4].

Bundlers and Paymasters. Bundlers are off-chain actors that collect `UserOperations` from an alternative mempool, simulate their validation, and submit batches to the `EntryPoint` as regular Ethereum transactions. Paymasters are optional contracts that can sponsor gas fees on behalf of users, enabling gasless transactions where a third party covers execution costs [4].

Counterfactual deployment. Wallets need not be deployed before their first use. If a `UserOperation` includes an `initCode` field, the `EntryPoint` calls the specified factory contract to deploy the wallet at a deterministic address computed via `CREATE2` [20] before processing the operation. This means a wallet address can be known and funded in advance, with deployment deferred to the moment of first use [4].

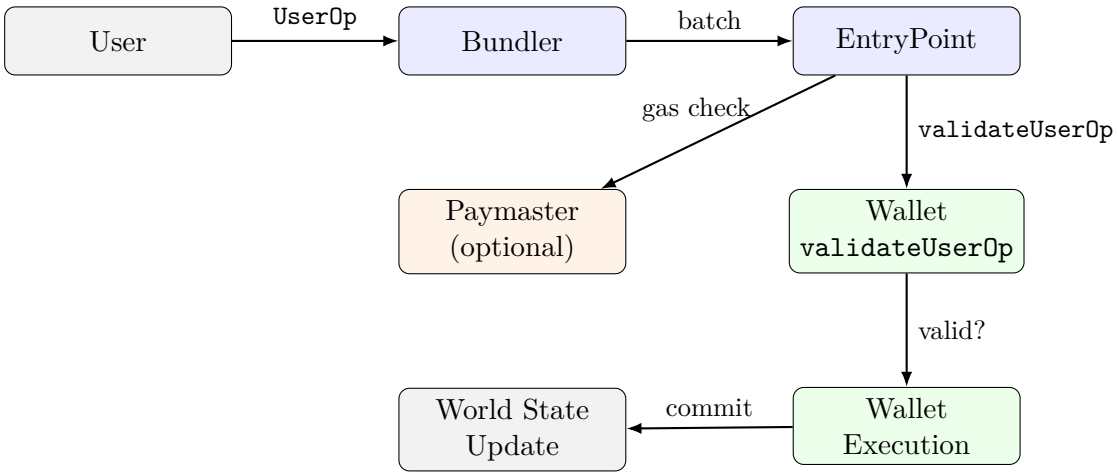


Figure 2: ERC-4337 UserOperation lifecycle. The bundler collects UserOperations and submits them to the EntryPoint, which calls `validateUserOp` on the wallet contract before dispatching execution. All custom authentication logic lives inside `validateUserOp`. Figure 2 contrasts with fig. 1 in one key respect: the validation step is no longer a fixed `ecrecover` call inside the protocol, but an arbitrary function call into a user-deployed contract.

3.2 Elliptic Curve Cryptography

Elliptic curve cryptography (ECC) is a family of public-key cryptographic mechanisms discovered in 1985 by Neal Koblitz and Victor Miller. ECC offers equivalent security to classical public-key schemes but with significantly smaller key sizes, making it faster and more efficient and therefore well suited for blockchain operations. Its security is based on the Elliptic Curve Discrete Logarithm Problem (ECDLP), introduced below. All three authentication mechanisms used in this thesis – Ethereum's native secp256k1, WebAuthn's secp256r1, and the Lithuanian eID's secp384r1 – are instances of the same underlying construction: ECDSA over different curve parameters. This section develops the mathematical basis shared by all three before the following subsections treat each curve individually.

3.2.1 Prerequisites: Finite Fields and Abelian Groups

The security and functionality of ECC are built upon the arithmetic of finite fields and the algebraic structure of abelian groups.

Finite fields. A *field* is a set $\mathbb{F}$ equipped with addition $(+)$ and multiplication $(\cdot)$, satisfying the following properties: $(\mathbb{F}, +)$ must be an abelian group with identity 0; $(\mathbb{F} \setminus \{0\}, \cdot)$ must be an abelian group with identity 1; and the distributive law $a \cdot (b + c) = a \cdot b + a \cdot c$ must hold. A finite field exists if and only if its order q – the number of elements it contains – is a prime power $q = p^m$, where p is a prime called the *characteristic* of the field [21]. The characteristic determines which form of the elliptic curve equation applies. This thesis exclusively concerns prime fields $\mathbb{F}_q$ where $q = p$ is prime and $p > 3$; all three curves used – secp256k1, secp256r1, and secp384r1 – are defined over fields of this form.

Abelian groups. An abelian group $(\mathbb{G}, +)$ is a set $\mathbb{G}$ with a binary operation satisfying associativity, commutativity, the existence of an identity element e, and the existence of an inverse for every element. A cyclic subgroup can be formed from the group by finding an element $g \in \mathbb{G}$; the smallest positive integer t such that $tg = e$ (in additive notation) is the *order* of the element g. The set of all multiples of g forms a cyclic subgroup $\langle g \rangle$ [21].

3.2.2 Elliptic Curve over a Finite Field

An elliptic curve E defined over a field $\mathbb{F}$ is the set of solutions to the general Weierstrass equation:

$$E : y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6 \tag{1}$$

where coefficients $a_i \in \mathbb{F}$ and the discriminant $\Delta \neq 0$. The condition $\Delta \neq 0$, where for the short Weierstrass form

$$\Delta = -16(4a^3 + 27b^2), \tag{2}$$

ensures the curve is smooth, meaning there are no points with multiple distinct tangent lines [21]. The smoothness of the curve is the fundamental requirement for the set of points on the curve to form an abelian group [21].

By applying an "admissible change of variables" [21], the general Weierstrass equation can be simplified considerably. For prime fields $\mathbb{F}_p$ with $p > 3$, the equation reduces to the short Weierstrass form:

$$E : y^2 = x^3 + ax + b, \quad a, b \in \mathbb{F}_p. \tag{3}$$

The set of rational points $E(\mathbb{F})$ consists of all pairs (x, y) satisfying the equation, together with a special *point at infinity* $\mathcal{O}$. This set $E(\mathbb{F})$ forms an additive abelian group in which $\mathcal{O}$ serves as the identity element [22].

The group law. The addition of two distinct points $P, Q \in E(\mathbb{F})$ is defined geometrically: draw the line through P and Q, find its third intersection with the curve, and reflect that point across the x-axis to obtain $P + Q$. When $P = Q$, the tangent line at P is used instead, giving the *point doubling* operation. These operations are computed algebraically using modular arithmetic over $\mathbb{F}_p$ [21, 22]. The *scalar multiple* dP for a positive integer d is defined as P added to itself d times via repeated point doubling and addition, and is the fundamental operation underlying all ECC constructions.

3.2.3 Elliptic Curve Cryptography

Domain Parameters. To use ECC in practice, all parties must agree on a common set of domain parameters $D = (q, FR, S, a, b, P, n, h)$ [21]:

- q: the field order;
- FR: the field representation used for elements;
- S: a seed used if the curve was generated verifiably at random;
- a, b: coefficients defining the curve equation;
- $P = (x_P, y_P)$: the base point;
- n: the prime order of P;
- h: the cofactor, defined as $h = \#E(\mathbb{F}_q)/n$.

Since this thesis exclusively concerns prime fields $\mathbb{F}_p$, the field representation FR is always the standard integer representation modulo p and carries no additional information – it is therefore omitted from the tuple in subsequent sections. Similarly, S is only present when the curve coefficients were generated verifiably at random from a public seed; for

curves with deterministically chosen parameters (such as secp256k1), S is absent. The shortened tuple used throughout this thesis is thus $D = (p, a, b, P, n, h)$, extended to $D = (p, a, b, S, P, n, h)$ where a seed is part of the curve specification.

Key Pairs. Given domain parameters, an entity's *private key* is an integer d selected uniformly at random from $[1, n-1]$. The corresponding *public key* is the curve point $Q = dP$, computed by scalar multiplication of the base point by d [22].

The Elliptic Curve Discrete Logarithm Problem. The security of ECC relies on the intractability of the *Elliptic Curve Discrete Logarithm Problem* (ECDLP): given an elliptic curve E, a base point P of prime order n, and a point $Q \in \langle P \rangle$, find the integer $d \in [0, n-1]$ such that $Q = dP$ [21]. Computing $Q = dP$ via repeated point addition is efficient; inverting this operation to recover d from Q and P is believed to be computationally infeasible for properly chosen curves and key sizes. All security guarantees of ECDSA rest on this hardness assumption.

3.2.4 Elliptic Curve Digital Signature Algorithm

ECDSA provides data origin authentication and data integrity. It requires a cryptographically secure hash function H to produce a message digest, and operates over domain parameters $D = (q, FR, S, a, b, P, n, h)$ agreed upon by all parties [21].

Signature Generation. To sign a message m, the signer performs the following steps [21, 22]:

1. Select a random per-message secret $k \in [1, n-1]$.
2. Compute $kP = (x_1, y_1)$ and convert the field element x_1 to an integer $\bar{x}_1$.
3. Compute $r = \bar{x}_1 \bmod n$. If $r = 0$, return to step 1.
4. Compute the hash $e = H(m)$.
5. Compute $s = k^{-1}(e + dr) \bmod n$. If $s = 0$, return to step 1.
6. The signature is the pair (r, s).

Signature Verification. To verify a signature (r, s) for message m under public key Q, the verifier performs the following steps [21, 22]:

1. Check that $r, s \in [1, n-1]$; reject otherwise.
2. Compute $e = H(m)$.

3. Compute the modular inverse $w = s^{-1} \bmod n$.

4. Compute $u_1 = ew \bmod n$ and $u_2 = rw \bmod n$.

5. Compute the point $X = u_1 P + u_2 Q$.

6. If $X = \mathcal{O}$, reject the signature.

7. Convert the x-coordinate x_1 of X to an integer $\bar{x}_1$ and compute $v = \bar{x}_1 \bmod n$.

8. Accept the signature if and only if $v = r$.

Correctness. We show that if (r, s) was generated according to the signature generation algorithm, then the verification algorithm accepts, i.e. $v = r$ [21].

From step 5 of signature generation:

$$s \equiv k^{-1}(e + dr) \pmod{n}. \tag{4}$$

Rearranging for k and expanding with $w \equiv s^{-1} \pmod{n}$, $u_1 \equiv ew \pmod{n}$, $u_2 \equiv rw \pmod{n}$:

$$k \equiv s^{-1}(e + dr) \equiv we + wrd \equiv u_1 + u_2 d \pmod{n}. \tag{5}$$

Substituting $Q = dP$ into the point computed in step 5 of verification:

$$X = u_1 P + u_2 Q = u_1 P + u_2(dP) = (u_1 + u_2 d)P \overset{(5)}{=} kP. \tag{6}$$

Since $X = kP$, the x-coordinate of X reduced modulo n equals r, so $v = r$ and verification accepts [21].

Security. The secret nonce k must be chosen uniformly at random and used only once. If the same k is used to generate signatures (r, s_1) and (r, s_2) for two different messages m_1 and m_2 with hashes e_1 and e_2, the two signature equations are:

$$s_1 \equiv k^{-1}(e_1 + dr) \pmod{n}, \tag{7}$$

$$s_2 \equiv k^{-1}(e_2 + dr) \pmod{n}. \tag{8}$$

Subtracting these equations, an observer can recover the nonce:

$$k \equiv (s_1 - s_2)^{-1}(e_1 - e_2) \pmod{n}. \tag{9}$$

Once k is known, the private key d follows directly from the signature equation:

$$d \equiv r^{-1}(ks - e) \pmod{n}. \tag{10}$$

Nonce reuse therefore results in complete compromise of the private key [21].

Public Key Recovery. A property of ECDSA is that the signer's public key can be recovered from a signature (r, s) and the signed message alone, without it being transmitted separately [23]. This is formalised in SEC 1 as the *Public Key Recovery Operation* [23].

Given domain parameters $D = (q, a, b, P, n, h)$, a message m, and a valid ECDSA signature (r, s), the public key Q is recovered as follows [23]:

1. For $j = 0, \ldots, h$:
   (a) Set $x = r + jn$.
   (b) Reconstruct a candidate curve point R with x-coordinate x using the curve equation $y^2 = x^3 + ax + b \pmod{p}$. If no such point exists, continue to the next j.
   (c) If $nR \neq \mathcal{O}$, continue to the next j.
   (d) Compute $e = H(m)$.
   (e) For each candidate R and $-R$, compute:

$$Q = r^{-1}(sR - eP) \pmod{n}. \tag{11}$$

   (f) Verify that Q is the authentic public key. If so, output Q.
2. If no valid Q is found, output `invalid`.

In practice, the cofactor $h = 1$ for all three curves used in this thesis, so only $j = 0$ is needed. The two candidate points R and $-R$ correspond to the two solutions y and $p - y$ of the curve equation for a given x-coordinate. The recovery identifier $v \in \{0, 1\}$ indicates which of the two candidates is correct, removing the ambiguity without additional verification [23]. Ethereum encodes this as the triple (v, r, s) appended to every signed transaction.

3.2.5 secp256k1 – Ethereum Native Signatures

secp256k1 is the elliptic curve used by both Bitcoin and Ethereum for all transaction signing. It is a Koblitz curve standardised by the Standards for Efficient Cryptography Group (SECG) in SEC 2 [24]. Before Bitcoin adopted it, secp256k1 was virtually unused; Ethereum inherited the same choice directly from Bitcoin.

Domain parameters. The domain parameters $D = (p, a, b, P, n, h)$ of secp256k1 are specified in SEC 2, Section 2.4.1 [24]:

- $p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1$, a pseudo-Mersenne prime;
- $a = 0$, $b = 7$, giving the short Weierstrass equation

$$E : y^2 \equiv x^3 + 7 \pmod{p}; \tag{12}$$

- $G = (G_x, G_y)$, the base point specified in SEC 2 [24];
- n, the prime order of G, a 256-bit integer close to p;
- $h = 1$, the cofactor.

Key pair generation. Key pair generation for secp256k1 follows the procedure in SEC 1, Section 3.2.1 [23]. Given domain parameters D:

1. Select a private key d uniformly at random from $[1, n - 1]$.
2. Compute the public key $Q = dG$.
3. Output the key pair (d, Q).

In Ethereum, the account address is derived from Q as the rightmost 20 bytes of Keccak-256$(Q_x \| Q_y)$ [18], where where Keccak-256 is the cryptographic hash function standardised as SHA-3 [25], Q_x and Q_y are the 32-byte big-endian encodings of the public key coordinates.

ECDSA signing and verification. Signing and verification for secp256k1 follow the ECDSA scheme defined in SEC 1, Sections 4.1.3 and 4.1.4 [23], instantiated with domain parameters T and hash function $H =$ SHA-256 (or Keccak-256 in Ethereum's case). The algorithms are as defined in section 3.2.4, with base point $P = G$ and group order n.

Ethereum message signing and `ecrecover`. When signing a message off-chain, Ethereum wallets do not sign the raw message m directly. Instead, EIP-191 [26] mandates that a fixed prefix be prepended before hashing, producing the digest:

$$z = \text{Keccak-256}(\texttt{"\textbackslash x19Ethereum Signed Message:\textbackslash n"} \| \text{len}(m) \| m)\,. \tag{13}$$

The signer then produces an ECDSA signature (v, r, s) over z using their private key d, following SEC 1, Section 4.1.3 [23]. The `0x19` prefix is intentionally chosen to make the signed payload invalid as an RLP-encoded Ethereum transaction, preventing a signed message from being replayed as a transaction [26].

Verification proceeds by recovering the signer's address from (v, r, s) and z using the public key recovery operation defined in SEC 1, Section 4.1.6 [23], and comparing the result to the expected address. The EVM exposes this as the `ecrecover` precompile at address `0x01` [18]:

1. Compute $z = \text{Keccak-256}(\texttt{prefix} \| \text{len}(m) \| m)$ as in eq. (13).
2. Call $\texttt{ecrecover}(z, v, r, s)$, which runs the SEC 1 recovery operation [23] and returns the 20-byte Ethereum address of the recovered public key Q, or zero address on failure.
3. Accept the signature if the returned address matches the expected signer.

3.2.6 secp256r1 – WebAuthn Signatures

secp256r1 is a 256-bit prime field elliptic curve jointly standardised by NIST and SECG [27, 24]. Its parameters were generated verifiably at random from the seed `C49D360886E704936A6678E1139D26B7819F7E90` using the ANSI X9.62 procedure – the "r" in secp256r1 denotes this random generation [24]. The domain parameters $D = (p, a, b, G, n, h)$ are specified in SEC 2, Section 2.4.2 [24]:

- $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$;
- $a \equiv -3 \pmod{p}$, chosen to admit a faster point doubling formula [27];
- $b =$ `0x5ac635d8aa3a93e7b3ebbd55769886bc651d06b0cc53b0f63bce3c3e27d2604b`, a constant derived from the generation seed S [24];
- $G = (G_x, G_y)$: the base point specified in SEC 2 [24];
- n: the prime order of G;
- $h = 1$: the cofactor.

These parameters give the short Weierstrass equation:

$$E : y^2 \equiv x^3 - 3x + b \pmod{p}. \tag{14}$$

P-256 provides approximately 128 bits of security [27] and is natively supported by most hardware security modules, smartphone secure enclaves, and smartcards. It is the mandatory curve in the WebAuthn standard [28].

3.2.7 WebAuthn procedure, parties and terminology.

The W3C Web Authentication specification [28] defines three parties involved in every operation:

Relying Party (RP). The web application that wishes to authenticate the user. It is identified by a *RP ID*, which is a valid domain string. The RP is responsible for generating challenges, storing public keys, and verifying assertions [28].

Authenticator. The hardware or software entity that holds the private key and performs cryptographic operations on behalf of the user, subject to user consent [28]. The specification distinguishes two kinds: a *platform authenticator* is built into the device (e.g. Apple Touch ID, Android fingerprint sensor, Windows Hello), while a *roaming authenticator* is an external device (e.g. a YubiKey) that communicates over USB or NFC. In both cases the private key never leaves the authenticator.

User Agent (Client). The browser. It mediates all communication between the RP and the authenticator, enforces the origin and RP ID scoping rules, and provides the JavaScript API (`navigator.credentials`) [28]. JavaScript running on a page cannot override the origin value the browser writes into signed data – this is a browser-enforced invariant.

A *credential* in WebAuthn is a secp256r1 key pair scoped to a single RP ID and stored inside an authenticator. The credential is identified by a *credential ID*, an opaque byte string chosen by the authenticator at registration time [28].

Registration ceremony. The *registration ceremony* creates a new credential and delivers the public key to the RP [28]. The exchange proceeds as follows:

1. The RP generates a fresh random *challenge* $c \leftarrow \{0,1\}^{128}$ and sends it to the browser together with the RP ID and user account information.

2. The browser calls `navigator.credentials.create(options)`, prompting the user for consent (biometric, PIN, or touch).

3. The authenticator generates a fresh P-256 key pair (d, Q) scoped to the RP ID, stores d internally, and produces:

   - `authData`: a byte string containing the *rpIdHash* (= SHA-256(RP ID)), a flags byte encoding user presence (UP) and user verification (UV), a 32-bit *signCount* initialised to zero, the *AAGUID* identifying the authenticator model, the credential ID, and the public key Q [28];

- an *attestation statement* providing cryptographic evidence of the authenticator's properties [28].

4. The browser returns `authData` and the attestation statement to the RP.

5. The RP verifies the attestation, checks that the `rpIdHash` matches SHA-256 of its own RP ID, and stores the credential ID and public key Q associated with the user account [28].

Authentication ceremony. The *authentication ceremony* produces a signed assertion proving that the user controls the private key registered in a prior registration ceremony [28]:

1. The RP generates a fresh challenge c and sends it to the browser together with the credential ID.

2. The browser calls `navigator.credentials.get(options)`, prompting the user for consent.

3. The authenticator looks up the credential by ID, verifies that the requested RP ID matches the one stored at registration, and increments the signCount. It then produces:

   - `authData`: the same structure as above, now with the updated signCount and without the credential ID and public key fields [28];
   - `clientDataJSON`: a JSON object written by the *browser* (not JavaScript) containing the ceremony type `"webauthn.get"`, the base64url-encoded challenge c, and the page origin [28];
   - a P-256 ECDSA signature over

$$m = \texttt{authData} \parallel \text{SHA-256}(\texttt{clientDataJSON}). \tag{15}$$

4. The browser returns `authData`, `clientDataJSON`, and the signature to the RP.

5. The RP verifies [28]:

   (a) the `rpIdHash` in `authData` equals SHA-256 of its own RP ID;

   (b) the `origin` in `clientDataJSON` matches the expected origin;

   (c) the `challenge` in `clientDataJSON` matches the one it issued;

   (d) the signCount has increased since the last ceremony;

   (e) the P-256 signature over m verifies under the stored Q.

3.2.8 secp384r1 – Lithuanian eID Signatures

secp384r1 is a 384-bit prime field elliptic curve jointly standardised by NIST and SECG [27, 24]. Like secp256r1, its parameters were generated verifiably at random from a public seed using the ANSI X9.62 procedure [24]. It provides approximately 192 bits of security – a full security level above P-256 [27].

Domain parameters. The domain parameters $D = (p, a, b, S, P, n, h)$ of secp384r1 are specified in SEC 2, Section 2.5.1 [24]:

- $p = 2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$, a pseudo-Mersenne prime;
- $a \equiv -3 \pmod{p}$, chosen to admit a faster point doubling formula [27];
- $b =$ `0xb3312fa7e23ee7e4988e056b e3f82d19181d9c6efe8141120314088f 5013875ac656398d8a2ed19d2a85c8edd3ec2aef`, a constant derived from the generation seed S [24];
- $S =$ `A335926AA319A27A1D00896A6773A4827ACDAC73`, the public seed from which a and b were derived [24];
- $G = (G_x, G_y)$: the base point specified in SEC 2 [24];
- n: the prime order of G;
- $h = 1$: the cofactor.

These parameters give the short Weierstrass equation:

$$E : y^2 \equiv x^3 - 3x + b \pmod{p}. \tag{16}$$

The Lithuanian eID card. The Lithuanian eID card is a hardware security device issued by the Migration Department [29]. The secp384r1 private key is generated on the card during personalisation and is non-exportable by design – no interface exists to read the raw key material. All signing operations are performed internally by the card's secure element. Access to the signing functionality is protected by a PIN: the card requires the correct PIN to be presented before it will perform any cryptographic operation, and permanently locks after a fixed number of consecutive incorrect attempts [29]. This means that physical possession of the card alone is insufficient to produce a valid signature – an attacker who steals the card still cannot sign without the PIN.

X.509 certificates and DER encoding. The eID card does not just store a raw key pair – it stores a full X.509 certificate issued and signed by the Migration Department PKI [29]. An X.509 certificate is a DER-encoded ASN.1 structure that binds a public key to an identity by carrying the subject's attributes like name, personal number and gender alongside the public key and a CA signature over the entire structure [30]. DER (Distinguished Encoding Rules) is a binary, canonical serialisation of ASN.1 that ensures a unique byte representation for any given certificate, which is essential for signature verification: the CA signs the DER-encoded `TBSCertificate` field, and any verifier must parse the same bytes to reproduce the signed message [30].

When the card performs a signing operation via the PKCS#11 interface, it also returns the resulting ECDSA signature in DER encoding – an ASN.1 sequence containing two integer values r and s, with leading bytes added where necessary to indicate positive values, length of (r, s) and type bytes.

3.3 Zero-Knowledge Proofs

Zero-knowledge proofs were introduced by Goldwasser, Micali, and Rackoff [31] as a cryptographic protocol between a *prover* and a *verifier* in which the prover convinces the verifier that a statement is true without revealing anything else. A proof system for a relation $\mathcal{R}$, where a *statement* x and a *witness* w, where the witness w is the private input that the prover knows but does not wish to disclose, satisfy $(x, w) \in \mathcal{R}$, must satisfy three properties [32]:

Completeness. Given a statement and a witness, an honest prover can convince the verifier.

Soundness. A malicious prover cannot convince the verifier of a false statement.

Zero-knowledge. The proof does not reveal anything but the truth of the statement – in particular it does not reveal the prover's witness.

Interactive proofs and their limitations. The original formulation of zero-knowledge proofs was *interactive* [31]: the verifier sends random challenges and the prover responds over multiple rounds, with each round reducing the probability that a cheating prover could have guessed correctly. While this achieves negligible soundness error, it requires the verifier and prover to be online simultaneously and exchange multiple messages. For blockchain use cases, which are relevant to us, this is very impractical – a smart contract executes atomically and cannot wait for a prover to respond to challenges. What we need is a proof that can be generated entirely off-chain and verified on-chain in a single transaction.

Non-interactive proofs via Fiat–Shamir. The *Fiat–Shamir heuristic* solves the interactivity problem by replacing the verifier's random challenges with the output of a cryptographic hash function applied to the prover's prior messages [33]. Since a hash function behaves as a random oracle, the prover cannot predict or influence the challenges, preserving soundness. The result is a *non-interactive* proof: a single self-contained object generated entirely by the prover, which any verifier can check independently without any interaction. In the random oracle model, the Fiat–Shamir transform preserves both soundness and zero-knowledge [34].

From statements to general computation. Non-interactivity solves the blockchain deployment problem, but a second challenge remains: the kinds of statements we want to prove – verifying a secp384r1 signature, parsing an ASN.1 certificate, checking an age constraint – are complex computations, not simple algebraic relations. We need a proof system that can handle arbitrary computation. The standard approach is to encode the computation as a system of arithmetic constraints over a prime field $\mathbb{F}_p$, called a *Rank-1 Constraint System* (R1CS) [34].

Standard non-interactive zero-knowledge proofs can in principle handle any NP statement, but their proof size grows linearly with the size of the computation, making them impractical for the complex circuits we require. The solution is to demand *succinctness*: a proof whose size is constant or logarithmic in the circuit, regardless of how many constraints it contains. Proof systems with this additional property are called zkSNARKs – *Succinct Non-interactive ARguments of Knowledge* – and they are what modern blockchain applications use in practice. Groth16, for instance, produces a proof consisting of exactly three elliptic curve group elements irrespective of circuit size [32].

An R1CS over $\mathbb{F}_p$ is a system of m constraints of the form:

$$\langle \mathbf{a}_i, \mathbf{z} \rangle \cdot \langle \mathbf{b}_i, \mathbf{z} \rangle = \langle \mathbf{c}_i, \mathbf{z} \rangle, \quad i = 1, \ldots, m, \tag{17}$$

where $\mathbf{z} \in \mathbb{F}_p^n$ is the *witness vector* containing the public inputs, private inputs, and all intermediate values, and $\mathbf{a}_i, \mathbf{b}_i, \mathbf{c}_i \in \mathbb{F}_p^n$ are fixed coefficient vectors that encode the computation [34]. Any computation – including elliptic curve arithmetic, hash functions, and comparisons – can be decomposed into a sequence of addition and multiplication gates, each of which becomes one R1CS constraint. A valid witness $\mathbf{z}$ satisfies all m constraints simultaneously, certifying that the computation was performed correctly.

zkSNARKs. With an R1CS encoding of the computation in hand, we can state precisely what a zkSNARK requires beyond completeness, soundness, and zero-knowledge [34]:

Succinctness. The proof size and verification time are both sublinear in the number of constraints m – usually a few hundred bytes and milliseconds of verification

regardless of how large the circuit is.

Non-interactivity. As previously said, the proof is a single message requiring no interaction between prover and verifier.

Argument of knowledge. The proof system is an *argument* rather than a proof: soundness holds against computationally bounded cheating provers. Knowledge soundness additionally guarantees that any prover producing an accepting proof must "know" a valid witness – formally, there exists an efficient *extractor* that, given oracle access to the prover, outputs a valid witness [34].

The succinctness property is what makes zkSNARKs practical for blockchains: no matter how expensive the computation being proved the on-chain verification cost remains constant.

3.3.1 Groth16

Groth16, proposed by Jens Groth in 2016 [32], is the specific zkSNARK construction used in this thesis. It achieves the smallest proof size and fastest verification of any known general-purpose zkSNARK, which is why it is the dominant scheme for on-chain verification in Ethereum applications. Its one drawback – a circuit-specific trusted setup – is acceptable for this thesis because the age verification circuit is fixed at deployment.

Trusted setup. Groth16 requires a one-time *trusted setup* that produces a *structured reference string* (SRS) encoding the circuit's structure as elliptic curve group elements [32]. The setup involves sampling toxic randomness $(\alpha, \beta, \gamma, \delta, \tau) \in \mathbb{F}_p^5$; the SRS is derived from these values and made public, but the values themselves must be permanently discarded. If any participant retains their contribution, they can use it to forge proofs for false statements – creating a proof that an invalid certificate satisfies the age constraint, for example. This is mitigated through a *multi-party computation ceremony* in which the SRS is secure as long as at least one participant honestly discards their randomness [32].

Proof structure. A Groth16 proof consists of exactly three elliptic curve group elements (A, B, C) where $A, C \in \mathbb{G}_1$ and $B \in \mathbb{G}_2$, totalling approximately 192 bytes on the BN254 curve used by Ethereum [32]. This size is constant regardless of the number of R1CS constraints.

Verification. The verifier accepts the proof if and only if the following pairing equation holds [32]:

$$e(A, B) = e(\alpha, \beta) \cdot e\left(\sum_{i=0}^{l} a_i \cdot \frac{\gamma_i}{\gamma}, \gamma\right) \cdot e(C, \delta), \tag{18}$$

where $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is a bilinear pairing, $a_0, \ldots, a_l$ are the public inputs, and $\alpha, \beta, \gamma, \delta$ are elements of the SRS. The equation checks that the prover's group elements are consistent with both the SRS and the public inputs, without revealing anything about the private witness.

3.3.2 Circom and SnarkJS

The R1CS encoding and Groth16 proof generation are handled in this thesis by two tools: Circom and SnarkJS.

Circom is a domain-specific language for writing arithmetic circuits [35]. A Circom circuit describes the computation to be proved – in this thesis, certificate parsing, P-384 signature verification, and the age constraint – as a hierarchy of *templates*, which are reusable parameterised components that declare input, output, and intermediate signals along with the quadratic constraints $A \cdot B = C$ that relate them. The Circom compiler translates the circuit into an R1CS file and a witness generation program. The number of constraints in the resulting R1CS determines both proving time and the size of the trusted setup.

SnarkJS is the JavaScript toolchain that executes the Groth16 trusted setup, generates proofs given a witness, and exports a Solidity verifier contract from the verification key [36]. The setup proceeds in two phases: Phase 1 (*Powers of Tau*) is a universal, circuit-independent ceremony whose output can be reused across different circuits; Phase 2 is circuit-specific and produces the proving key and verification key from the Phase 1 output and the R1CS. The exported Solidity contract exposes a `verifyProof(proof, publicInputs)` function that executes the pairing check in eq. (18) on-chain using the EVM's `bn256` precompiles.

4 Implementation of Smart Accounts

This chapter presents the design and implementation of the five-level progressive wallet system. Each level is described in terms of its smart contract logic, the off-chain components required to interact with it, and the authentication flow end to end. Each section also introduces an architecture diagram that is extended at every level, making it easy to see what changes between each implementation, what new benefits to the user are added, and what infrastructure is shared.

All five levels share the same core infrastructure: an account factory contract that deploys account instances via `CREATE2` which computes a deterministic account address from the constructor parameters, the ERC-4337 `EntryPoint` contract as the transaction coordinator which is called for every transaction on submission. The `validateUserOp` function is the main point of variation across levels – everything above and below it will

be mostly identical, except for the last implementation.

4.1 Basic ECDSA Account

Level 1 establishes the baseline. The wallet authenticates `UserOperation`s using a standard secp256k1 ECDSA signature, replicating the security model of an EOA but within the ERC-4337 framework. This level serves as the reference point against which all subsequent levels are measured in terms of gas cost and added capability.

Contract design. The wallet contract implements `validateUserOp` by recovering the signer's address from the `userOpHash` and the (v, r, s) components packed into the `signature` field, using the EVM's `ecrecover` precompile. If the recovered address matches the stored owner address, the operation is accepted. The owner address is set once during deployment via an `initialize` function called by the factory.

Factory and counterfactual deployment. The wallet factory deploys new wallet instances as minimal proxies pointing to a shared implementation contract, using `CREATE2` with the owner's secp256k1 public key as the salt. This allows the wallet address to be computed deterministically before deployment – the user can receive funds at their wallet address before the wallet contract exists on-chain, with deployment deferred to the first `UserOperation`.

Authentication flow. The frontend computes the `userOpHash`, signs it with the owner's private key using standard secp256k1 ECDSA, packs (v, r, s) into the `signature` field, and submits the `UserOperation` to the `EntryPoint`. The `EntryPoint` calls `validateUserOp` of the account, which runs `ecrecover` and compares the result to the stored owner.

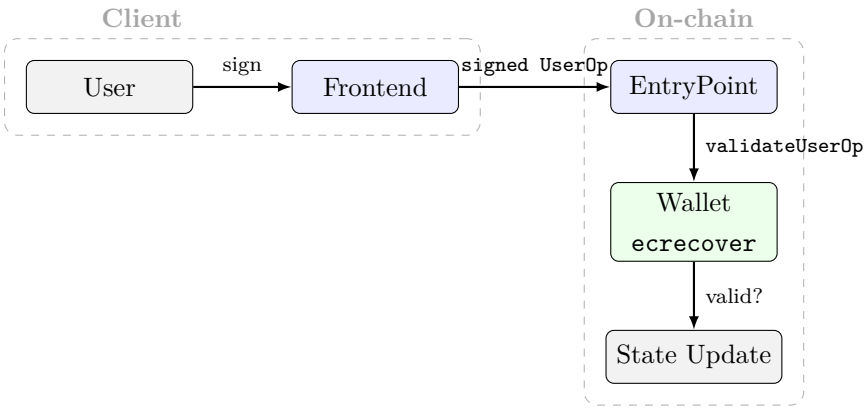


Figure 3: Level 1 authentication flow. The user signs the `userOpHash` with their secp256k1 private key. The wallet recovers the signer address via `ecrecover` and accepts if it matches the stored owner.

4.2 WebAuthn Biometric Account

Level 2 replaces the secp256r1 private key with a WebAuthn passkey, allowing users to authorise transactions using the same biometric authenticator – fingerprint, face unlock, or hardware key – they use to unlock their device. The private key never leaves the secure hardware, the user interacts only with a native authentication prompt.

Contract design. The smart account stores a secp256r1 public key (Q_x, Q_y) as two `bytes256` values set during initialisation. The `validateUserOp` implementation reconstructs the signed WebAuthn message – the concatenation of `authData` and the SHA-256 hash of `clientDataJSON` – from the fields packed into the `signature` payload, verifies that the `clientDataJSON` contains the `userOpHash` as its challenge, and then verifies the secp256r1 ECDSA signature using the OpenZeppelin secp256r1 library [6]. The challenge binding check is essential: without it, a valid WebAuthn assertion for one operation could be replayed against another.

Frontend registration and signing flow. During registration, the frontend calls the WebAuthn API to create a new credential, extracts the secp256r1 public key from the authenticator response, and passes it to the factory to deploy a wallet initialised with that key. During signing, the frontend constructs a `UserOperation` with the `userOpHash` set as the WebAuthn challenge, calls browser's `navigator.credentials.get()` to trigger the authenticator, and packs the resulting `authData`, `clientDataJSON`, and (r, s) signature components into the `signature` field.

What this level adds. The replacement of classical ECDSA signing mechanism greatly improves usability for the user – to submit a transaction it's enough to scan a fingerprint or face. The user does not need to securely store his private key or remember a seed phrase. All of this is handled by the secure enclave where the WebAuthn key is stored.

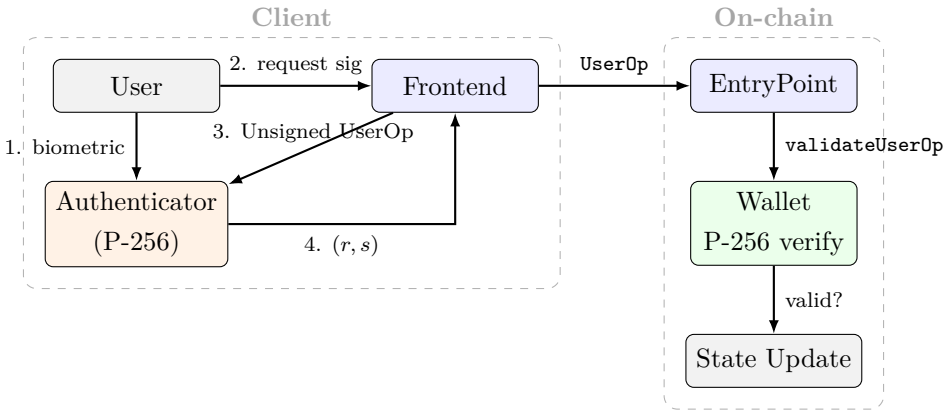


Figure 4: Level 2 authentication flow, extending Level 1. The secp256k1 signature is replaced by a WebAuthn secp256r1 assertion produced by the device authenticator. User builds a `UserOp` and signs via WebAuthn biometric authenticator, which returns the signature of `UserOp`. After submitting to `EntryPoint` wallet verifies the secp256r1 signature and challenge binding on-chain using the OpenZeppelin secp256r1 library.

4.3 Lithuanian eID Account

Level 3 replaces the device authenticator with a physical Lithuanian eID card. The user approves a transaction by inserting their card and entering their PIN, producing a secp384r1 ECDSA signature that is verified on-chain against the public key extracted from their government-issued certificate.

Contract design. The wallet stores the eID certificate's secp384r1 public key as two `bytes256` values during initialisation. The `validateUserOp` implementation passes the `userOpHash`, the DER-decoded signature, and the stored public key to the Ethstonia Curve384 Solidity library [13], which performs the full secp384r1 ECDSA verification.

Backend and PKCS#11 bridge. Interacting with the eID card requires a software stack that is not available in the browser. A Python backend receives the `UserOp` from the frontend and forwards it to a Python helper process running under Rosetta 2 (required to interface with the x86_64 mCard Toolbox library on Apple Silicon). The Python process opens a PKCS#11 session, locates the authentication certificate and private key slot, requests a secp384r1 ECDSA signature over the hash, and returns the DER-encoded signature. The Python backend DER-decodes the signature into (r, s) bytes and returns it to the frontend, which the frontend packs into the `UserOperation`'s `signature` field.

What this level adds. By having the keys from the digital eID certificate in the smart contract, the account achieves a level of compliance typically required by the governments. They are now able to associate a smart account to a person's identity by mapping these keys in the contract to their database entries.

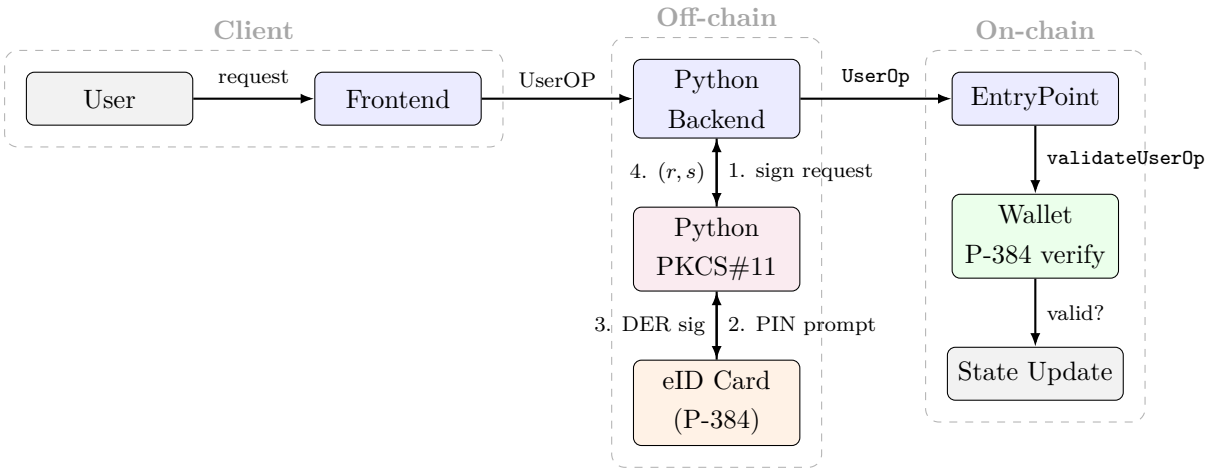


Figure 5: Level 3 authentication flow, extending Level 1. A Python backend and PKCS#11 bridge communicate with the physical eID card to produce a secp384r1 DER signature. This signature gets decoded into (r, s) in the backend to match the expected format of the verifier. The wallet verifies the signature on-chain using the Ethstonia library.

4.4 Adding Zero-Knowledge Age Verification to the eID Account

Level 4 introduces zero-knowledge age verification. Rather than revealing any personal data, the user proves on-chain that they are over 18 by submitting a Groth16 zkSNARK proof derived from their government-signed eID certificate. The contract verifies the proof without learning anything about the user's actual birth date or any other certificate attribute. This section first presents the formal design and security arguments for the proof system, then describes the implementation.

4.4.1 Problem Statement and Design Goals

Let cert denote a DER-encoded X.509 certificate issued and signed by the Lithuanian government CA with public key $\mathsf{pk}_{\mathsf{CA}}$. The certificate contains, among other fields, the holder's personal code, from which the date of birth dob can be extracted. Let d_{now} denote the current date. We wish to construct a proof system for the following statement:

> *"I know a certificate* cert *such that: (1) the government CA signature over* cert *verifies under* $\mathsf{pk}_{\mathsf{CA}}$*, and (2) the date of birth* dob *encoded in* cert *satisfies* $d_{\mathsf{now}} - \mathsf{dob} \geq 18$ *years."*

The public inputs to the proof are $\mathsf{pk}_{\mathsf{CA}}$ and d_{now}. The private witness is the full certificate cert, including dob and all other personal attributes. The proof must satisfy:

Completeness. Any honest user holding a valid government-issued certificate with $d_{\mathsf{now}} - \mathsf{dob} \geq 18$ years can produce an accepting proof.

Soundness. No prover can produce an accepting proof for a certificate that is not signed by the government CA, or for a birth date that does not satisfy the age threshold.

Zero-knowledge. The verifier learns nothing from the proof beyond the fact that the statement is true – in particular, neither `dob` nor any other personal attribute is revealed.

The soundness property is what distinguishes this design from naive ZK age verification systems that accept a user-supplied birth year as a private input. By binding the proof to a government CA signature verified *inside* the circuit, any attempt to supply a false birth date produces a certificate that fails the signature check, making the proof reject.

4.4.2 Certificate Parsing in the Circuit

An X.509 certificate in DER encoding is a nested ASN.1 structure following the TLV (Type-Length-Value) format. The outermost sequence contains three fields: the `TBSCertificate` (the signed payload), the signature algorithm identifier, and the CA's signature. Within `TBSCertificate`, the subject field contains the holder's personal code as the `serialNumber` attribute, for example `50305100121`. The personal code encodes the date of birth in digits 2–7 in the format `YYMMDD`, with the first digit indicating the century and gender: `5` denotes a male born in the 21st century, giving a birth date of 2003/05/10. The circuit extracts `dob` by slicing this six-digit substring and parsing it arithmetically. The certificate is signed by the Migration Department CA using P-384 ECDSA with SHA-384. Note that the same date of birth can also be extracted from the `id-pda-dateOfBirth` field.

Certificate structure. The following excerpt from a real Lithuanian eID certificate illustrates the structure described above [29]:

```
Subject: C=LT, CN=SIMONAS KANAPECKAS, SN=KANAPECKAS,
         GN=SIMONAS, serialNumber=50305100121
Subject Public Key Info:
    Public Key Algorithm: id-ecPublicKey
        Public-Key: (384 bit)
        ASN1 OID: secp384r1
        NIST CURVE: P-384
Signature Algorithm: ecdsa-with-SHA384
Issuer: C=LT, O=Migracijos departamentas prie LR VRM, CN=MD CA
X509v3 Subject Directory Attributes:
    id-pda-dateOfBirth: 20030510120000Z
    id-pda-gender: M
    id-pda-countryOfCitizenship: LT
```

Parsing this structure inside an arithmetic circuit is non-trivial because ASN.1 lengths are variable-width and the certificate layout is not fixed. The `asn1-parser-circom`

library [16] provides Circom templates that traverse the TLV structure by tracking byte offsets as circuit signals. Given the DER bytes as an array of field elements, the parser outputs the start and end indices of each tagged field. The circuit then slices the `serialNumber` bytes at the identified indices and extracts the date of birth substring for arithmetic comparison.

4.4.3 secp384r1 Signature Verification in the Circuit

The CA signature over the `TBSCertificate` is a secp384r1 ECDSA signature. Verifying it inside the circuit requires implementing the full ECDSA verification algorithm – scalar multiplication on the secp384r1 curve – as a system of R1CS constraints.

Let the `TBSCertificate` bytes be M, the CA public key be $Q_{\mathsf{CA}} = (q_x, q_y)$, and the signature be (r, s). The circuit constrains the following:

1. Compute $z = \text{SHA-384}(M) \bmod n$, where n is the P-384 group order.
2. Compute $w = s^{-1} \bmod n$.
3. Compute $u_1 = zw \bmod n$ and $u_2 = rw \bmod n$.
4. Compute $(x_1, y_1) = u_1 G + u_2 Q_{\mathsf{CA}}$ using P-384 point arithmetic.
5. Constrain $x_1 \equiv r \pmod{n}$.

The `ecdsa-p384-circom` library [17] provides the P-384 curve arithmetic templates. Each field element in the circuit must be represented as a multi-limb integer because P-384 values exceed the BN254 scalar field size used by Groth16, requiring careful range checking to prevent overflow across limbs. SHA-384 is implemented as a separate Circom template using standard bit manipulation constraints.

4.4.4 Age Constraint

Once dob has been extracted and the certificate signature verified, the circuit must constrain that the holder is at least 18 years old. Let dob and d_{now} both be encoded as integers of the form YYYYMMDD. The age in years is not directly computable by simple subtraction due to the non-uniform length of months, so the circuit uses the following decomposition:

$$\mathsf{age} = \mathsf{year}(d_{\mathsf{now}}) - \mathsf{year}(\mathsf{dob}) - \mathbf{1}\Big[\mathsf{mmdd}(d_{\mathsf{now}}) < \mathsf{mmdd}(\mathsf{dob})\Big] \tag{19}$$

where $\mathsf{year}(\cdot)$ extracts the four-digit year, $\mathsf{mmdd}(\cdot)$ extracts the month-day suffix, and $\mathbf{1}[\cdot]$ is the indicator function that equals 1 if the birthday has not yet occurred in the current year. The circuit decomposes dob and d_{now} into year, month, and day components, computes eq. (19), and constrains $\mathsf{age} \geq 18$ using a standard Circom range check template.

4.4.5 Soundness and Zero-Knowledge Arguments

Soundness. The soundness of the overall proof follows from the soundness of the Groth16 proof system and the collision resistance of SHA-384. Suppose a cheating prover attempts to produce an accepting proof for a false statement – either a certificate not signed by the CA, or a birth date that does not satisfy the age threshold. In the first case, the secp384e1 signature verification constraints in the circuit will not be satisfiable: the prover would need to find a witness (r, s, Q_{CA}) such that the ECDSA verification equations hold for a message M not signed by the CA, which requires solving the ECDLP on secp384r1. In the second case, the age constraint $\mathsf{age} \geq 18$ will not be satisfiable for the extracted birth date. In both cases, no valid R1CS witness exists, and by the soundness of Groth16, no proof can be generated that the verifier accepts – except with negligible probability bounded by the Groth16 knowledge soundness error [32].

Zero-knowledge. The zero-knowledge property follows from the zero-knowledge of Groth16. The proof consists of three elliptic curve points that are computationally indistinguishable from random group elements to any probabilistic polynomial time adversary who does not know the proving key trapdoor [32]. The public inputs – $\mathsf{pk}_{\mathsf{CA}}$ and d_{now} – are known to the verifier by assumption and reveal nothing about the witness. In particular, dob, the subject name, and all other personal attributes encoded in cert remain information-theoretically hidden from the on-chain verifier.

4.4.6 Trusted Setup and Verifier Contract

The circuit is compiled with Circom and the resulting R1CS is fed into SnarkJS for the Groth16 trusted setup. Phase 1 reuses a publicly available Powers of Tau ceremony output sufficient for the circuit's constraint count; Phase 2 is circuit-specific and produces a proving key and verification key. SnarkJS exports a Solidity verifier contract from the verification key, which is deployed once and referenced by the wallet contract. The wallet's `validateUserOp` calls the verifier contract's `verifyProof` function with the proof and public inputs extracted from the `signature` field.

What this level adds. This level introduces an additional compliance constraint while not impacting the privacy of the user. Most of the regulations demand that financial services like cryptocurrencies are only used by adults. This ensures that only a person over 18 can use such an account but does not disclose his date of birth.

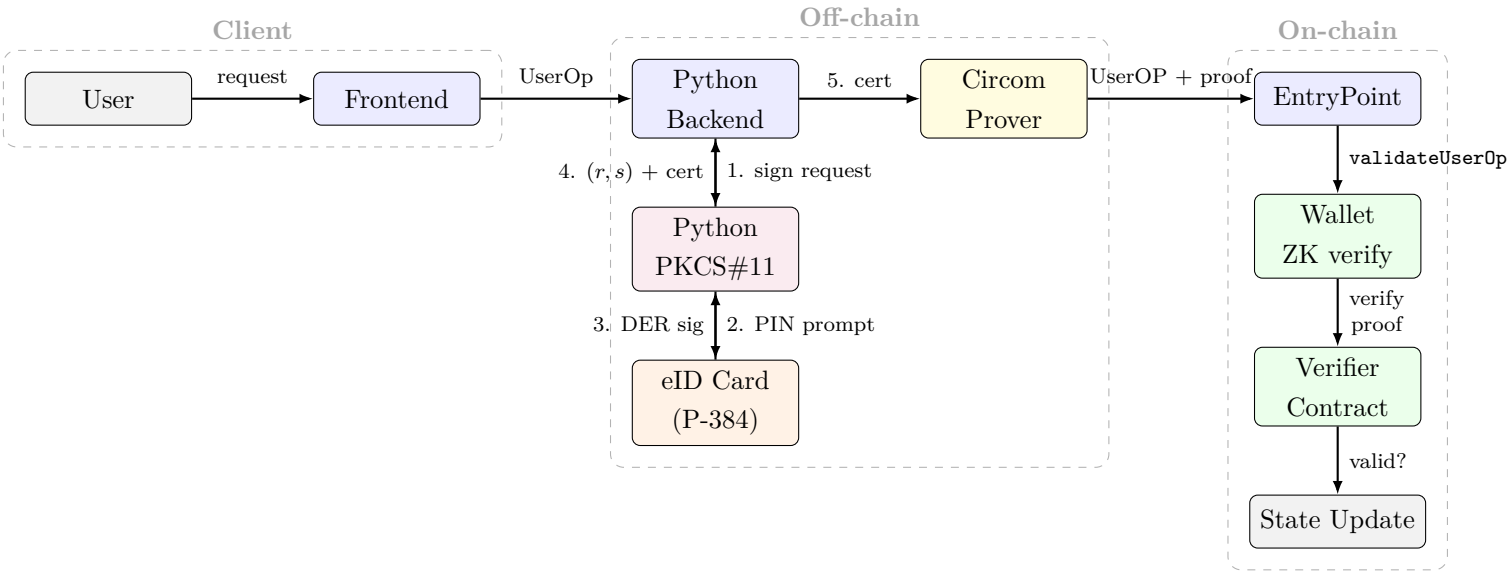


Figure 6: Level 4 authentication flow. The eID certificate is retrieved via the PKCS#11 bridge and passed to the Circom prover off-chain. The Groth16 circuit verifies the government CA's P-384 ECDSA signature over the certificate and checks the age constraint inside the proof. The wallet delegates proof verification to a dedicated Groth16 verifier contract, which checks the proof against the CA's public key and current date supplied as public inputs. Once compliance is established, subsequent transaction authentication proceeds identically to Level 3: each transaction is authorised by an on-chain secp384r1 signature verification.

4.5 Combined Final Multi-Factor Account

Level 5 integrates all prior authentication mechanisms, except for secp256k1, into a single wallet that enforces age compliance at deployment and routes each subsequent operation to the appropriate verification path. The ZK age proof is a deployment prerequisite: the wallet factory accepts a Groth16 proof as a constructor argument and only deploys the wallet if the proof is valid. This means age compliance is baked into the wallet's existence; an account that exists on-chain is by definition owned by someone who was verified to be over 18 at deployment time.

Deployment with age verification. The factory contract's `createAccount` function takes the standard arguments (WebAuthn public key, eID public key) plus a Groth16 proof and its public inputs. Before deploying the wallet, the factory calls the verifier contract's `verifyProof` function. If the proof is invalid the deployment reverts; if valid, the wallet is deployed via `CREATE2` with the verified keys stored in contract storage.

Operation routing in `validateUserOp`. Once deployed, the wallet routes each `UserOperation` based on an *operation type* byte prepended to the `signature` field:

- **Type 0 – WebAuthn**: the remainder of the signature is a packed WebAuthn assertion. The wallet verifies the P-256 signature and challenge binding as in Level 2.
- **Type 1 – eID recovery**: the remainder contains a P-384 DER-encoded signature from the eID card plus the new WebAuthn public key to install. The wallet verifies the P-384 signature against the stored eID public key, and if valid, overwrites the stored WebAuthn key with the new one.

Key storage layout. The wallet stores the WebAuthn secp256r1 public key, the eID secp384r1 public key, and the Groth16 age proof in contract storage. Storing the proof on-chain means that anyone can independently verify that the wallet owner was over 18 at deployment time by calling the verifier contract directly. The eID key is permanent and cannot be replaced – it is the trust anchor of the wallet. The WebAuthn key can be replaced by presenting a valid eID P-384 signature, allowing recovery if the WebAuthn credential is lost or the device is replaced.

What this level adds. This level gives us the best of all worlds. Usability via easy-to-use WebAuthn signatures and the ability to recover lost keys via the secp384r1 eID signature. At the same time, this account ensures a level of compliance by associating an account to a real person via the public key form eID in the contract, as well as privately ensuring that he is eligible to use this service by being over 18 years old.

Deployment and transaction flows. In ERC-4337, deployment and transaction submission are not separate operations — both are initiated by submitting a `UserOperation` to the `EntryPoint`. If no contract exists at the sender address, the `EntryPoint` calls the factory to deploy the wallet first, then proceeds with execution as normal. In Levels 1 through 4 this distinction was not shown explicitly because the initialisation and regular transaction submission take the same parameters and follow the same flow — the wallet either gets deployed or it does not, but the diagram is identical in both cases. At Level 5, the first `UserOperation` carries significantly more: the WebAuthn public key, the eID public key, a Groth16 ZK age proof, and an eID P-384 signature over the WebAuthn public key — binding the two keys together under the government identity. The factory verifies the ZK proof and the eID signature before deploying. Every subsequent `UserOperation` follows the same flow as Level 2: the user signs with their WebAuthn authenticator and the wallet verifies the P-256 signature, with the eID path available as a recovery mechanism. The two diagrams below separate these flows for clarity.

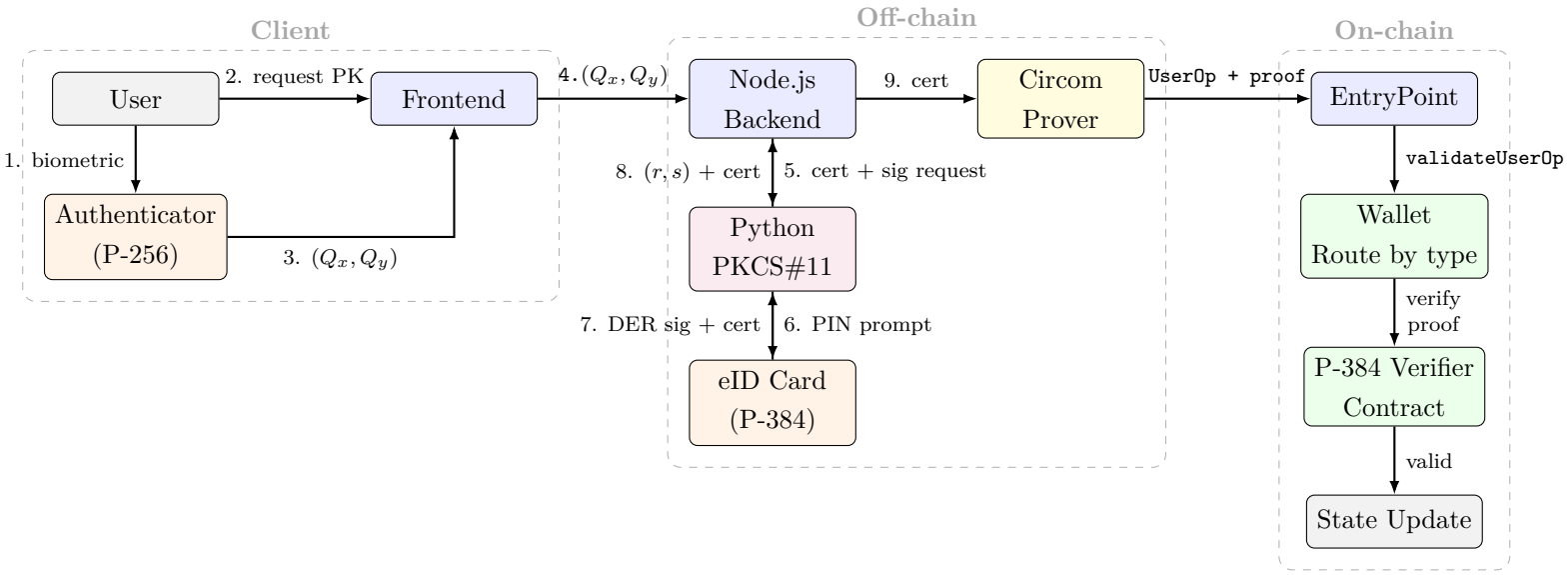


Figure 7: Level 5 initialization flow. The user generates WebAuthn keys, requests the eID certificate and signs the first `UserOp` containing the WebAuthn keys $Q = (Q_x, Q_y)$ in the data field with the eID. The ZK age proof is generated as in Level 4. The proof, eID public keys are passed as additional arguments. They are saved into immutable variables of the smart account upon initialization.

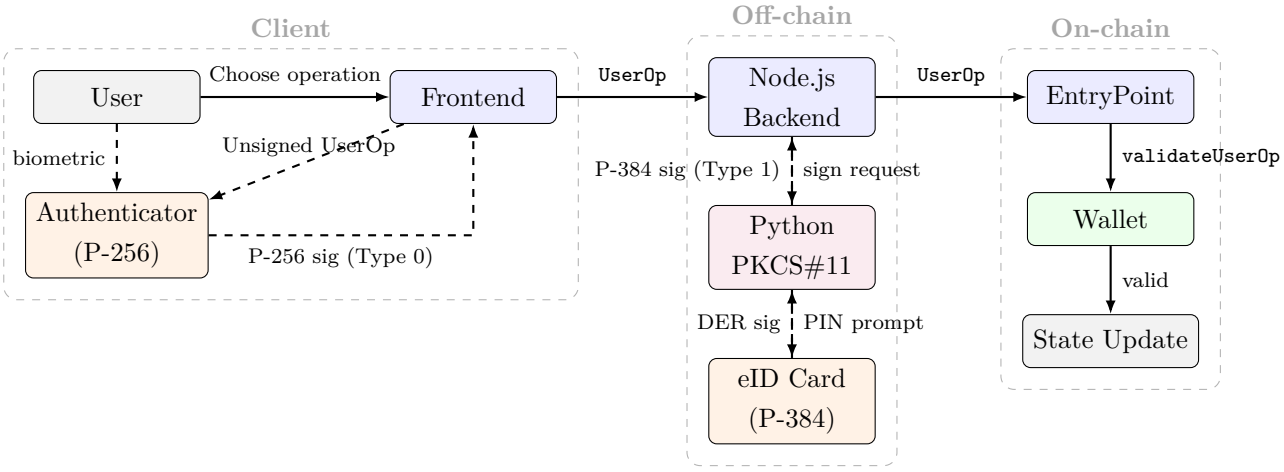


Figure 8: Level 5 transaction routing. The user chooses an operation he wants to perform: submit a regular transaction signed by secp256r1 or submit a key recovery transaction signed by secp384r1. The wallet inspects the operation type byte prepended to the `signature` field. Type 0 carries a WebAuthn P-256 assertion for daily transactions. Type 1 carries an eID P-384 signature for key recovery, replacing the stored WebAuthn key if the signature verifies. Dashed arrows show the respective signing paths.

5 Evaluation

5.1 Testing Environment

All five wallet levels were deployed and tested on a local Hardhat development network. Levels 1 and 2 operate within Ethereum mainnet's block gas limit and could in principle be deployed directly to a public testnet. Levels 3, 4 and 5 include secp384r1 on-chain signature verification, whose gas cost – even after the precomputed lookup table optimisation from the Ethstonia library [13] – exceeds the current mainnet block gas limit of approximately 30 million gas. These levels were therefore tested with the Hardhat gas limit set to unlimited. All gas measurements were taken at the `validateUserOp` call level, which is the primary on-chain cost for each level.

5.2 Evaluation Metric: Gas Cost

Gas is the unit of computational work on the Ethereum Virtual Machine. Every EVM opcode has a fixed gas cost, and the total gas consumed by a transaction determines its fee. For smart contract wallets, the dominant cost is the `validateUserOp` call, which runs the authentication logic on every transaction. Gas cost is therefore the primary metric for evaluating the practicality of each authentication level: it determines whether the wallet can operate within the mainnet block gas limit, and what fee a user must pay per transaction.

The five levels use fundamentally different cryptographic primitives, so their gas costs differ by orders of magnitude. Level 1 uses the `ecrecover` precompile, which has a fixed cost of 3 000 gas [18]. Level 2 uses a pure-Solidity secp256r1 verifier [6]. Level 3 uses the Ethstonia secp384r1 verifier with precomputed lookup tables [13]. Level 4 uses a Groth16 zkSNARK verifier operating on BN254 pairing precompiles; its verification cost follows the formula [37]:

$$\mathrm{Gas}_{\mathrm{Groth16}} \approx 207\,000 + 7\,160 \times \ell, \tag{20}$$

where ℓ is the number of public inputs. Level 5 combines Level 2 for daily transactions and Level 3 for eID-based key recovery, with a ZK proof required at deployment time.

5.3 Results

table 1 summarises the `validateUserOp` gas cost for each level. Levels 3 and 5 (P-384 paths) exceed the mainnet block gas limit and were measured only in the local unlimited-gas Hardhat environment.

Level	Authentication	Gas (approx.)
1	secp256k1 ECDSA (`ecrecover`)	3 000
2	P-256 WebAuthn (Solidity verifier)	∼270 000–330 000
3	P-384 eID (lookup table optimised)	∼20 000 000
4	Groth16 ZK age proof ($\ell = 2$)	∼221 000
5	WebAuthn (daily) / P-384 (recovery)	∼270 000–330 000 / ∼20 000 000

Table 1: Approximate `validateUserOp` gas cost per authentication level. Level 1 uses the native `ecrecover` precompile at 3 000 gas [18]. Level 2 is based on the OpenZeppelin P-256 implementation [6]. Level 3 reflects the Ethstonia optimised P-384 verifier after precomputed lookup table reduction [13]. Level 4 is computed from eq. (20) with $\ell = 2$ public inputs (CA public key and current date) [37]. Level 5 costs depend on the operation type invoked.

Discussion. The results illustrate the core trade-off of the progressive system. Level 1 is the cheapest possible authentication at 3 000 gas, but provides no recovery mechanism and no compliance capability beyond what a standard EOA offers. Level 2 adds hardware-backed biometric authentication at a cost of approximately 270 000–330 000 gas per transaction – roughly 90–110 times more expensive than Level 1, but still well within the mainnet block gas limit and practical for everyday use. Level 4 adds age-compliance via zero-knowledge proof at approximately 221 000 gas per verification – actually *cheaper* than Level 2's WebAuthn verification, and required only once to establish compliance. This is because Groth16 verification reduces to three BN254 pairing operations via dedicated EVM precompiles, which are highly optimised. Level 3 and the P-384 recovery path in Level 5 remain the bottleneck: at approximately 20 million gas, on-chain secp384r1 verification is currently impractical on Ethereum mainnet but viable on L2 networks with higher gas limits or cheaper gas prices. The Ethstonia team notes that further optimisations – Jacobi coordinates and base-4 or more precomputes – could bring this below the mainnet block gas limit [13]. An alternative path is to wrap the secp384r1 transaction signature itself in a SNARK, as Level 4 already does for the CA certificate signature. The on-chain verifier would then check only a constant-size Groth16 proof rather than perform curve arithmetic directly, bringing the cost to the same order as Level 2; the SNARK need not provide privacy – succinctness alone is the goal. It is worth noting that a government could also subsidise such expensive transactions via ERC-4337 Paymasters, if it wished to incentivise transparent and auditable smart account usage on public blockchain networks.

5.4 Limitations

5.4.1 Identity Binding at Account Creation

The ZK age proof verified at deployment time proves that *someone* holding a valid eID certificate over 18 authorised the account creation, but it does not bind the account to the identity of the WebAuthn key holder. A user could in principle deploy a Level 5 account using their own eID proof but register a WebAuthn key belonging to another person, effectively creating an account that passes age verification but is controlled by a different party. This is a fundamental limitation of the design: the proof and the operational key are independent. In practice this is discouraged rather than prevented – the eID certificate used in the proof is linked to a real government-issued identity, meaning that any malicious activity traceable to the account can be investigated by authorities who can identify the certificate holder. A stricter construction could expose a public commitment to the certificate holder's unique identity – for example, by requiring the circuit to output $H(id)$, where id is the personal code extracted from the certificate and H is a collision-resistant hash function. The contract could then reject any deployment attempt that reuses a commitment already registered on-chain, preventing a single person from creating multiple compliant accounts while revealing nothing about the underlying identity beyond the commitment itself. The threat model therefore relies on accountability rather than technical enforcement for this specific attack vector, though the commitment approach described above would provide a stronger guarantee without sacrificing privacy.

5.4.2 Recovery Dependency on eID Card Availability

The eID card is the only recovery mechanism for a lost or compromised WebAuthn key. If both the WebAuthn credential and the eID card are lost simultaneously – for example, a stolen device and a stolen wallet – the smart account becomes permanently inaccessible. Whether the Lithuanian government retains any copy of the eID private key is not publicly documented. The card is described as generating keys during personalisation, but the policy on whether the issuing authority retains a copy is not disclosed. If no institutional copy exists, there is no fallback for key recovery. This reflects the fundamental tension between self-custody and recoverability: the stronger the guarantee that a private key cannot be extracted from the card, the stronger the trust anchor – but also the greater the risk of permanent loss.

6 Conclusion

This thesis set out to answer whether ERC-4337 account abstraction is expressive enough to encode government-grade authentication and privacy-preserving compliance directly at

the wallet level, without any off-chain intermediary. The answer is yes – with significant caveats about gas cost.

The five-level progressive wallet framework demonstrates that a single, coherent smart account architecture can incorporate secp256k1 ECDSA for baseline compatibility, WebAuthn biometric authentication for usability, Lithuanian government eID card signatures for identity compliance, and a zero-knowledge proof of age derived from a government-signed certificate for privacy-preserving compliance. Each level builds on the previous one without breaking the ERC-4337 abstraction boundary: all custom authentication logic lives inside `validateUserOp`, and the `EntryPoint` contract and account factory infrastructure are shared across all five levels unchanged.

The gas analysis reveals a clear stratification. Levels 1, 2, and 4 are viable on Ethereum mainnet today, with verification costs of 3 000, ∼330 000, and ∼221 000 gas respectively. The zero-knowledge proof verification at Level 4 is particularly encouraging: at ∼221 000 gas it is cheaper than WebAuthn verification, because Groth16 reduces to pairing operations on dedicated EVM precompiles. The bottleneck is P-384 verification at Level 3 and the recovery path at Level 5, which at ∼20 million gas currently exceeds the mainnet block gas limit. This is not a fundamental barrier – the Ethstonia team's further optimisations with Jacobi coordinates and extended precomputes are expected to bring P-384 verification within the block limit, and Layer 2 networks already provide sufficient headroom today.

Two security limitations temper the conclusions. First, the ZK age proof at deployment does not cryptographically bind the proving identity to the wallet's operational key, meaning the system relies on legal accountability rather than technical enforcement to prevent misuse. Second, the eID card is the only recovery mechanism for a lost WebAuthn key, and if both are lost simultaneously the account is permanently inaccessible – a tension inherent to hardware-anchored self-custody that no software design can fully resolve.

Taken together, the results show that privacy-preserving, government-grade identity verification is not only compatible with the ERC-4337 framework but can be delivered at gas costs that are practical for current EVM networks, at least for the ZK and WebAuthn paths. The remaining obstacle is P-384 on-chain verification, a problem that is engineering rather than fundamental, and one that existing optimisation work is actively closing.

References

[1] David Yaffe-Bellany. *FTX's Collapse: What We Know and What Comes Next*. 2022. URL: `https://www.nytimes.com/2022/11/11/business/ftx-bankruptcy.html`.

[2] Nathaniel Popper. “Lost Passwords Lock Millionaires Out of Their Bitcoin Fortunes”. In: (2021). URL: https://www.nytimes.com/2021/01/12/technology/bitcoin-passwords-wallets-fortunes.html.

[3] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. Ethereum Whitepaper. 2014. URL: https://ethereum.org/en/whitepaper/.

[4] Vitalik Buterin et al. *ERC-4337: Account Abstraction Using Alt Mempool*. Ethereum Improvement Proposal. 2021. URL: https://eips.ethereum.org/EIPS/eip-4337.

[5] Solidity Team. *Solidity Documentation*. https://docs.soliditylang.org. 2024.

[6] OpenZeppelin. *OpenZeppelin Contracts v5.1*. Open-source Solidity library. 2024. URL: https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/utils/cryptography/P256.sol.

[7] European Parliament and Council of the European Union. *Regulation (EU) 2024/1183 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework (eIDAS 2.0)*. Official Journal of the European Union, L 2024/1183, 30 April 2024. ELI: http://data.europa.eu/eli/reg/2024/1183/oj. 2024.

[8] European Parliament and Council of the European Union. *Regulation (EU) 2023/1114 on markets in crypto-assets, and amending Regulations (EU) No 1093/2010 and (EU) No 1095/2010 and Directives 2013/36/EU and (EU) 2019/1937 (MiCA)*. Official Journal of the European Union, L 150, pp. 40–205. ELI: http://data.europa.eu/eli/reg/2023/1114/oj. 2023.

[9] Vitalik Buterin. *EIP-86: Abstraction of Transaction Origin and Signature*. Ethereum Improvement Proposal. 2017. URL: https://eips.ethereum.org/EIPS/eip-86.

[10] Vitalik Buterin et al. *EIP-2938: Account Abstraction*. Ethereum Improvement Proposal. 2020. URL: https://eips.ethereum.org/EIPS/eip-2938.

[11] Micah Zoltu. *EIP-2718: Typed Transaction Envelope*. Ethereum Improvement Proposal. 2020. URL: https://eips.ethereum.org/EIPS/eip-2718.

[12] Ulaş Karaca, Yusuf Ergül, and Tayfun Yomralıoğlu. *RIP-7212: Precompile for secp256r1 Curve Support*. Rollup Improvement Proposal. 2023. URL: https://github.com/ethereum/RIPs/blob/master/RIPS/rip-7212.md.

[13] Leon Logvinov et al. *Ethstonia: Verify Estonian e-ID Signatures on Ethereum*. ETHBerlin Hackathon project. 2018. URL: https://github.com/LogvinovLeon/estid-sig.

[14] Polygon Technology. *Polygon ID: Zero-Knowledge Identity for Web3*. Product announcement. 2022. URL: https://polygon.technology/blog/introducing-polygon-id-zero-knowledge-own-your-identity-for-web3.

[15] ZKPassport. *ZKPassport: Privacy-Preserving Identity Verification*. Documentation. 2024. URL: https://docs.zkpassport.id/intro.

[16] zk-email contributors. *asn1-parser-circom: Circom ASN.1 Parser*. Open-source library. 2023. URL: https://github.com/zkemail/asn1-parser-circom.

[17] Crema Labs. *ecdsa-p384-circom: ECDSA P-384 Implementation for Circom*. Open-source library. 2023. URL: https://github.com/crema-labs/ecdsa-p384-circom.

[18] Gavin Wood. *Ethereum: A Secure Decentralised Generalised Transaction Ledger*. Yellow Paper. Ethereum Foundation, 2014. URL: https://ethereum.github.io/yellowpaper/paper.pdf.

[19] Vitalik Buterin. *EIP-155: Simple Replay Attack Protection*. Ethereum Improvement Proposal. 2016. URL: https://eips.ethereum.org/EIPS/eip-155.

[20] Vitalik Buterin. *EIP-1014: Skinny CREATE2*. Ethereum Improvement Proposal. 2018. URL: https://eips.ethereum.org/EIPS/eip-1014.

[21] Darrel Hankerson, Alfred Menezes, and Scott Vanstone. *Guide to Elliptic Curve Cryptography*. Springer, 2004. DOI: 10.1007/b97644.

[22] Don Johnson, Alfred Menezes, and Scott Vanstone. "The Elliptic Curve Digital Signature Algorithm (ECDSA)". In: *International Journal of Information Security* 1.1 (2001). DOI: 10.1007/s102070100002.

[23] Certicom Research. *SEC 1: Elliptic Curve Cryptography*. Tech. rep. Version 2.0. Standards for Efficient Cryptography Group, 2009. URL: https://secg.org/sec1-v2.pdf.

[24] Certicom Research. *SEC 2: Recommended Elliptic Curve Domain Parameters*. Tech. rep. Version 2.0. Standards for Efficient Cryptography Group, 2010. URL: https://secg.org/sec2-v2.pdf.

[25] National Institute of Standards and Technology. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. Tech. rep. FIPS PUB 202. NIST, 2015. URL: https://doi.org/10.6028/NIST.FIPS.202.

[26] Martin Holst Swende and Nick Johnson. *EIP-191: Signed Data Standard*. Ethereum Improvement Proposal. 2016. URL: https://eips.ethereum.org/EIPS/eip-191.

[27] National Institute of Standards and Technology. *Digital Signature Standard (DSS)*. FIPS Publication 186-5. NIST, 2023. DOI: 10.6028/NIST.FIPS.186-5.

[28] Dirk Balfanz et al. *Web Authentication: An API for Accessing Public Key Credentials Level 2*. W3C Recommendation. 2021. URL: https://www.w3.org/TR/webauthn-2/.

[29] Migration Department under the Ministry of Interior of the Republic of Lithuania. *Citizen Certificate Policy, v1.1*. Tech. rep. Ministry of Interior of the Republic of Lithuania, 2025. URL: https://www.nsc.vrm.lt/docs/MD_ATK_CP_v1.1_EN.pdf.

[30] David Cooper et al. *Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. RFC 5280. IETF, 2008. URL: https://www.rfc-editor.org/rfc/rfc5280.

[31] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. "The Knowledge Complexity of Interactive Proof Systems". In: *SIAM Journal on Computing* 18.1 (1989), pp. 186–208. DOI: 10.1137/0218012.

[32] Jens Groth. "On the Size of Pairing-Based Non-interactive Arguments". In: *Advances in Cryptology — EUROCRYPT 2016*. Vol. 9666. Lecture Notes in Computer Science. Springer, 2016, pp. 305–326. DOI: 10.1007/978-3-662-49896-5_11.

[33] Amos Fiat and Adi Shamir. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems". In: *Advances in Cryptology — CRYPTO 1986*. Vol. 263. Lecture Notes in Computer Science. Springer, 1987, pp. 186–194. DOI: 10.1007/3-540-47721-7_12.

[34] Justin Thaler. *Proofs, Arguments, and Zero-Knowledge*. Vol. 4. Foundations and Trends in Privacy and Security 2–4. Now Publishers, 2022. DOI: 10.1561/3300000030. URL: https://people.cs.georgetown.edu/jthaler/ProofsArgsAndZK.pdf.

[35] Marta Bellés-Muñoz et al. *Circom: A Robust and Scalable Language for Building Complex Zero-Knowledge Circuits*. Open-source project. 2022. URL: https://github.com/iden3/circom.

[36] iden3. *SnarkJS: zkSNARK Implementation in JavaScript*. Open-source library. 2020. URL: https://github.com/iden3/snarkjs.

[37] Kiwi. *Maximizing Efficiency in Ethereum ZKPs: Comparing Groth16 and FFLONK Gas Costs and Implementing Best Practices*. HackMD, Orbiter Research. 2023. URL: https://hackmd.io/@Orbiter-Research/S1nat__m0.